Novice developers often face challenges in addressing critical security aspects, which can lead to adverse consequences such as data leakage and reputational loss. This paper investigates the novice developers' security awareness and their security knowledge gap. We conducted semi-structured interviews with 32 Bangladeshi web developers of varying experience levels, which helped us to investigate the developers' security knowledge, learning paths, and implementation practices. Our findings reveal persistent gaps in the fundamental security concepts, because early insecure practices are often sustained in the code reviews and migrate into production systems. Furthermore, instead of traditional security education, developers across all experience levels primarily update their security knowledge when encountering concrete problems or human errors in their work

Web applications are the backbone of communications, business, education, and government services; however, they are still the most vulnerable to cyber attacks. Traditional web development training programmes often lack hands-on secure coding components due to time constraints, limited resources, and a curriculum that prioritises functionality over security [11]. Without proper guidance and foundational security knowledge, novice developers unknowingly create insecure applications by reusing unverified code snippets or relying on AI-based tools that may promote insecure coding practices [4][26]. As a result of these gaps in education and practice, novice developers tend to address security concerns only after vulnerabilities have already been exploited. Such threats, if not tackled properly, end up having consequences that are particularly noticeable in third-world countries like Bangladesh. In Bangladesh, from a test shown 64 % of government and business websites had severe vulnerabilities: SQL injection (56 %), XSS (53 %), and CSRF (15 %) [25]. Previous papers focused on quantifying vulnerabilities, but rarely have the developers themselves and their views, motives, obstacles, and knowledge gaps been substantively explored. To address this gap, the study adopts a qualitative approach using semi-structured interviews and thematic analysis to explore developers' security awareness, practices, and attitudes toward security training. It contributes to equity-focused computing by revealing the sociotechnical origins of insecure web development among novice developers in Bangladesh and offers insights to inform human-centered security education and policy interventions.

3 LITERATURE REVIEW

In this section, we have reviewed frequent web vulnerabilities such as SQL injection, Cross-Site scripting, Broken authentication, IDOR, Clickjacking, NoSQLi, DOM clobbering, CSP Bypass, CSS injection, and CDN Tampering. SQL Injection attacks the backend database. The significance of its growth from 100 reported cases to more than 7,000 is evident, and it is responsible for about 60% of all web vulnerabilities [12][18][13]. Cross-Site Scripting (XSS) is a frontend attack that exploits how user content is dynamically generated in frameworks. It is really important because it allows session hijacking and token theft at a massive scale; over 3.7 million form hijacking events were intercepted in one year. [9][19][23]. The Broken Authentication is a multi-layer attack that affects both the frontend and the backend. Misconfigured JWTs, OAuth flows, and RBAC or ABAC models can expose app vulnerabilities [9] [5] [32]. IDOR is a backend vulnerability that occurs when object IDs in URLs

are not being protected. The researchers demonstrated how simply changing the document ID in a parameter exposed payment records and acceptance letters on a live academic website[2][33]. Clickjacking is a purely frontend attack that uses invisible overlays and CSS manipulation to redirect user clicks. A survey of the top 500 websites found only 14% had any frame-busting protection at all [15][29]. NoSQL Injection attacks the backend, specifically MongoDB. When the user input is not sanitized, operators such as $ne and Boolean "always-true" conditions bypass authentication. Novice developers in particular are vulnerable, as they might perceive that the NoSQL databases are immune to any injection attack [28][14]. DOM Clobbering is a type of frontend attack that involves overwriting HTML IDs with JavaScript Globals, resulting in XSS or CSRF by code reuse.

The Hulk framework found 497 vulnerabilities in the Tranco Top 5,000, including zero-days in major libraries like Webpack and Google Client API [24][21]. CSP Bypass is a security failure in the

frontend, where poorly crafted Content Security Policies are made ineffective. A study of 26,011 real-world CSP rules found that 94.72% failed to block XSS, largely due to JSONP and AngularJS

API abuse [27][36]. CSS Injection attacks the frontend and may be overlooked because CSS is not considered a threat. It has been demonstrated to silently steal auto-filled credentials without any JavaScript, and 478 Android apps have been identified as vulnerable via CSS-based injection

[16][17]. CDN tampering impacts the frontend delivery layer and backend infrastructure. It is found that developers' lack of understanding of CDN configuration may lead to DoS or spoofing, which can be caused by forwarding-loop attacks, cache poisoning, or certificate mismanagement

[22][35][8].

4 RELATED WORK

Security vulnerabilities in software are frequently traced back to developer behavior rather than technical failures alone. Anu et al. [3] performed a lab study with developers working on vulnerable applications and discovered that security simply isn't a priority in a software development environment. Developers prioritize functionality over security while coding, and tend to assume common cases for their code. Tahaei and Vaniea [34] confirmed that security issues are consistently

overlooked in development processes, and developers lack the knowledge and tools needed to embed security in their day-to-day work. The persistence of insecure practices is well-documented.

Bai et al. [4] studied how insecure code from online forums spreads to real software, finding that

developers copy security-related code without properly assessing it. Alghamdi et al. [1] noted this

is particularly harmful for novice developers who lack sufficient knowledge to critically evaluate

code quality. Charoenwet et al. [7] analyzed more than 135,000 code review comments and found
that 30–36% of identified security issues were never resolved, often due to disagreements about
solutions (18–20%), allowing vulnerabilities to reach production systems. Perry et al. [26] further
showed that developers using AI coding assistants like GitHub Copilot tend to generate more insecure code and overestimate its security, making foundational security knowledge more critical
than ever.
These issues extend beyond the Western world. Kekulluoglu and Acar [20] found that security and privacy are deprioritized in Turkish startups due to lacking awareness, skills, and resources. Shrestha et al. [31] interviewed 15 Bangladeshi software developers and found that security is not
a concern in the local software industry, shaped by client pressure and a lack of security education.
Gasiba et al. [11] further highlighted a gap between industry needs and existing security education,
showing that professional training lacks proper coding instruction relevant to real development challenges. In this study, we focus on security awareness among web developers in Bangladesh,
with special attention on novice developers. We investigate how developers are trained, what security knowledge they acquire, and where gaps remain, while examining the mindset differences
between novice, mid-level, and senior developers in learning and applying security concepts in practice.

5 METHODS

This paper examines the perception of the security threat by the developers and the factors that contribute to insecure coding among novices. Semi-structured interviews were employed to elicit in-depth experiences and allow a free flow of discussion [10][37]. The subsequent sections elaborate
on the protocol development, participant recruitment and data analysis.

5.1 Research Design and Implementation

To minimize bias, developers were recruited through email and snowball sampling across various
companies and roles, and interviewed either in-person or online. Novices needed a minimum of three previous projects. The last sample was a group of 32 developers: 20 novices, 9 mid-level, and 3
senior-level developers, including web, embedded, mobile, and desktop development. The pilot test
involving three participants (novice, mid-level, senior) was used to refine the protocol by making it
clear and timely; pilot data were also retained because they did not affect validity.

5.2 Interview Process

Each one hour semi-structured interview was structured around the following sections:
Introduction. In order to be transparent and consistent, a brief description was provided.
Professional Background. Participants shared their experience, coding, code reviews, and how they learned to develop.
Security Practices and Mentorship. Senior developers shared their experience with security tooling, mentorship and some tips for novices.
Security-Specific Topics. The questions were designed for different levels: basic questions for Novices, questions regarding web vulnerabilities for Mid level, and advanced scenarios for Senior level Developers. The vulnerabilities that were covered were SQLi, XSS, IDOR, Broken Authentication, Clickjacking, NoSQLi, CSP Bypass, CDN Tampering, DOM Clobbering, and CSS Injection.

5.3 Data Analysis, Ethics & Limitations
Thematic analysis was used to analyse interviews [6]. The recordings were transcripted and the
participants were classified as novices (N1-20), mid-level (M1-9), and seniors (S1-3). The questions
varied according to developers' experience, so coding was performed independently per group by three independent researchers using Taguette, followed by collaborative cycles until thematic saturation [30]. The research was approved by the Institutional Review Board. There were no identifiable data that were gathered; all data were anonymised, stored securely, and will be destroyed
after the publication as per data privacy laws. Participation was voluntary, with informed consent being obtained and the right to withdrawal at any time. Some limitations includes sample (n=32) being skewed towards novices, and inadequate mid and senior level representation. Moreover, snowball sampling created selection bias among security-conscious individuals, and the emphasis
on common vulnerabilities hindered the coverage of advanced threats.
6 RESULTS & ANALYSIS
This section presents the findings of thematic analysis of the interviews with Novice, Mid and Senior
level developers, revealing their differences in security awareness, practices, and responsibility.
6.1 Novice Developer:
Security Awareness and Knowledge Gaps
Novice developers were largely unfamiliar with fundamental threats like SQL injection and XSS, and with frameworks like OWASP Top 10, with most responding "I have no idea, I have only heard
of these vulnerabilities." Security received minimal attention in formal education, with one noting "I have learned nothing (security-related) from university, just PHP and SQL".
Learning Sources and Framework Dependency
Novice developers acquire technical skills through self-directed learning—courses and YouTube,
with minimal structured security training. They rely heavily on AI-generated code—tools like

ChatGPT and BLACKBOX AI—with one admitting they “just used the code ChatGPT gave me”. University curricula provided only shallow security coverage, which students found unengaging and quickly forgot. Reliance on frameworks and libraries like Firebase creates knowledge gaps, as
developers fail to critically analyse security threats due to time constraints and a prioritisation of functionality over security.

Security Practices and Real World Experience

Novice developers treat security as an afterthought, not an integral part of their coding journey. They
default to familiar tools such as “Google Firebase”, “JSON Web Token”, “Multi-Factor Authentication”,
and “Password Hashing” without informed reasoning, and rarely participate in code reviews ranging
from “Never” to “Not often” leaving vulnerabilities undetected. Secure coding practices are minimal,
with some relying on “AI.” Limited threat awareness, deadline pressure, and a shortcut-driven mindset collectively undermine security.

6.2 Mid Developer:

Secure Development and Practices

Security tool adoption was inconsistent across teams. While some used automated tools like Jenkins,
Spring Boot, and dependency checking, others relied on manual processes, neglecting security during setup. M1 noted that developers are not initially security-conscious, with separate security
teams sometimes hired post-development. M9 reinforced this: “Our priority is to complete the features.

Only after that is security policies added.” M3 mentioned using reliable dependency versions to mitigate vulnerabilities, though not all projects applied this consistently. Overall, security remained
reactive rather than proactive across most teams.

Responsibility Sharing

Security responsibility was shared but varied by role: seniors, DevOps, and specialists set standards
while novices focused on functionality. M2 noted Security issues are not prioritized during development, mainly novice developers overlook security,” though issues were typically caught in code
reviews—If insecure code is overlooked. . . it is immediately fixed.” M9’s explanation reveals that role
specialization creates knowledge boundaries, limiting mid-level developers’ exposure to security tools. In the absence of dedicated specialists, developers were required to self-educate on security

best practices.

Vulnerability Awareness and Security Knowledge

Developers recognized common vulnerabilities like SQLi, XSS, and IDOR, but awareness varied.

For SQLi and XSS, most relied on frameworks rather than deep understanding — M2 explained, “Modern-day frameworks can mitigate this vulnerability (SQLi). . . people don’t inherently keep this vulnerability in mind,” M3 echoed, “SQL injection is one of the most well-known attacks, so current frameworks already mitigate it,” and M2 noted for XSS, “Developers don’t need to handle it as tools handle it” — yet M5 admitted, “I know SQL injection, but I am not familiar with the term Query Parameterization,” revealing a gap between awareness and understanding. For lesser-known threats, knowledge gaps were more pronounced: M6 dismissed DOM Clobbering as “Less risk,” others stated “Not sure,” M2 remarked “Frontend developers would know about this vulnerability” for CSS Injection, M4 noted it is “harder” to detect and that “CSP bypass sometimes works,” while for CDN Tampering and SRI, M2 stated “Junior and mid-level developers do not handle it; senior developers do” and M3 said “If there are vulnerabilities in CDN providers it will be fixed immediately. . . I trust it,” reflecting a blind spot in shared responsibility. Authentication choices were convention-driven — M2, M6, and M7 used Bcrypt, with M6 noting it “came as a default. . . I know how to use it” — while IDOR awareness was stronger, as M2 emphasized, “UUID is not sufficient; access control is required.”

6.3 Senior Developer:

Security Practices and Mentorship

Senior developers highlighted tension between deadlines and security, with S1 noting “Security practices are unspoken rules. . . security cannot be disregarded.” In many teams, seniors handle security and deployment before handing projects to juniors—as S3 explained, “Senior developers do not let junior developers handle initial project setups. . . security-related aspects are handled by senior developers. Only after these measures are in place is the project handed over to juniors.” Teams rely on established authentication libraries for routine tasks, while cybersecurity tools help identify vulnerabilities. Mentorship also emerged as critical, with S2 noting “Mentorship is necessary. . . They emphasize security and encourage novice developers to consider security measures.”

Security-Specific Topics

Senior developers identified SQLi and XSS as high-risk despite framework use. S1 noted API gate-
ways as an infrastructure-level mitigation for SQLi, while S2 stressed consistent practice, attributing
XSS mostly to novice developers neglecting frontend security. Complex attacks exposed further gaps: DOM Clobbering was largely overlooked, with S2 warning that predictable global variables
could be exploited. For CDN Tampering, developers favored trusted providers like Cloudflare, while

CSS Injection showed mixed awareness. For example, S1 had no backend exposure to it, and S2
recommended data sanitization and avoiding inline CSS. NoSQLi lacked universal protection, with
developers preferring managed services and libraries like mongoSanitize over traditional databases.
For Clickjacking, seniors relied on server-side techniques, rejected client-side frame busting, and
stressed beginner training.

7 DISCUSSION

The shift between a novice and a senior developer is a crucial change in the perception of security as
well as its implementation in professional practice. Novice developers typically enter the industry with substantial knowledge of functionality and software development practices but insufficient security awareness. Universities reinforce this by offering security as a distinct, often optional sub-
ject and positioning it as peripheral to core software development education. Mid-level developers
are halfway in between, working in high-stakes areas such as fintech and blockchain. They are generally proactive about basic security practices, but often lack expertise in advanced security threats. The pressure of the deadline makes security a deferred aspect, and the main quality control
mechanism is post-development code reviews. Senior developers have made security a fundamental
professional responsibility, learned on the job, and making critical early-stage security decisions before delegating the projects to juniors.
At all levels, the dependence on frameworks develops some blind spots. Novices believe that automated protections will take the place of security knowledge; mid-level developers will imple-
ment safeguards without understanding their limitations, leaving them vulnerable to advanced attack vectors and infrastructure-level issues such as CDN Tampering and Subresource Integrity,
which they pass on to DevOps or seniors. The senior developers are more aware of strategic

gaps, but they recognize the existence of gaps in the emerging and fast-changing threats. Security seriousness is directly proportional to the exposure to an incident, seniors who have faced breaches treat security as non-negotiable, while novices and many mid-level developers approach it through a compliance lens rather than a risk-aware mindset.

8 CONCLUSION & FUTURE WORK

This study examined security awareness, learning behaviors, and coding practices of 32 Bangladeshi web developers across three experience levels through semi-structured interviews. Findings reveal a progressive security knowledge gap: novices relied on framework defaults and AI-generated code with little understanding of vulnerabilities like SQLi, XSS, or the OWASP Top 10; mid-level developers implemented safeguards without deeper comprehension, leaving blind spots for threats like DOM Clobbering, NoSQLi, and CDN Tampering; and seniors approached security as a professional responsibility shaped by real-world experience rather than formal education. Across all groups, on-the-job learning proved more effective than academic instruction. Future work should address the sample's geographic and experience-level limitations by adopting cross-cultural, balanced participants, and should explore advanced vulnerabilities such as CDN Tampering, Subresource Integrity, and DOM Clobbering through more targeted interview protocols.